

Clarion Endpoint Context & Enforcement Extension

Architectural Specification Cross-platform process, application, AI-agent, and device-aware policy architecture.

Core Architecture Statement

Clarion should use Security Group Tags (SGTs) for **coarse device/session-level network segmentation**, while endpoint-native controls provide **process, application, and agent-level attribution and enforcement**. Clarion acts as the policy intelligence layer that correlates endpoint context, identity, network session, destination, behavior, and enforcement outcomes.

Primary Decision	Design Position
Can one access port have one SGT per running agent?	No. Multiple processes and agents can run simultaneously behind one endpoint/session.
Should endpoints inject Cisco CMD headers?	Not as the primary design. Use endpoint evidence to influence trusted policy through Clarion, ISE, and network enforcement.
Where should per-agent enforcement happen?	On the endpoint and/or at a proxy, API gateway, or model gateway.
Where should SGT enforcement happen?	At the device/session/fabric layer for coarse segmentation and trusted propagation.

Contents

1. Executive Summary
2. Problem Statement
3. Design Goals and Non-Goals
4. Core Architectural Principles
5. High-Level Architecture
6. Component Overview
7. Identity and Policy Model
8. SGT and TrustSec Strategy
9. Enforcement Architecture
10. AI Agent and General Application Detection
11. Data Flows and Scenarios
12. Cisco ISE and TrustSec Integration
13. Telemetry Sources
14. Data Model and APIs
15. Security and Trust Model
16. Deployment Model
17. MVP Phases
18. Engineering Backlog
19. Final Design Summary

1. Executive Summary

Clarion should not rely only on network-layer identity such as IP address, VLAN, subnet, or a single Security Group Tag assigned to an access port. These constructs are useful, but they are too coarse for modern endpoints where many users, applications, containers, AI agents, developer tools, browsers, scripts, and automation frameworks can run simultaneously on the same device.

This architecture extends Clarion with a **host-aware context and enforcement layer**. The endpoint layer provides process, application, and agent-level context. The network layer provides broad segmentation and propagation. The Clarion platform correlates the evidence and chooses the right enforcement point.

Clarion uses SGTs for coarse device/session-level network enforcement.

Clarion uses endpoint-native controls for fine-grained process, application, and agent-level enforcement.

Layer	Primary Question	Best Enforcement Fit
Endpoint	Which process or agent generated the flow?	Local firewall, WFP, eBPF, Network Extension, proxy redirect
Network Access	Which device/session is this?	ISE authorization, SGT, dACL, VLAN, VRF, CoA
Network Fabric	Which source group may reach which destination group?	SGACL, firewall policy, TrustSec inline propagation
Application/Gateway	What request/action is being performed?	API gateway, model gateway, SWG, CASB, DLP
Clarion	What is the full context and desired policy outcome?	Decision, orchestration, simulation, recommendation

2. Problem Statement

Traditional network access control assumes a simple model: one device, one user, one network session, and one access policy. Modern endpoints break that model. One access port may carry traffic from a browser, AI coding assistant, autonomous agent, Python runtime, PowerShell script, Docker container, local LLM, database client, cloud sync client, and background operating system service at the same time.

```
The network may see:
source IP  = 10.10.20.55
source MAC = AA:BB:CC:DD:EE:FF
source SGT = Developer_Device
```

```
Clarion needs to know:
python.exe launched by cursor.exe under user Steve
calling an external model API
after reading local source code
then attempting to access a production database.
```

That level of context cannot be solved by SGT alone. It requires a layered policy model that combines endpoint telemetry, network session identity, destination intelligence, and application-layer controls.

3. Design Goals and Non-Goals

3.1 Design Goals

- Complement Clarion with endpoint, process, application, and agent context.
- Support Linux, Windows, and macOS using platform-native telemetry and enforcement.
- Avoid treating SGT as per-process identity.
- Apply to AI agents and to general endpoint/application policy.
- Create a unified policy model across user, device, session, process, agent, flow, destination, and risk.

- Support multiple enforcement points: endpoint, network, firewall, proxy, API gateway, model gateway, SIEM/SOAR, and MDM/EDR.

3.2 Non-Goals

- Do not assume one SGT per process or AI agent.
- Do not inject Cisco TrustSec CMD headers from ordinary endpoints as the primary design.
- Do not trust the endpoint to self-assert its SGT.
- Do not replace ISE, EDR, MDM, firewalls, proxy/SASE, SIEM, or DLP.
- Do not quarantine an entire endpoint when a precise process-level block is sufficient.

4. Core Architectural Principles

4.1 Identity Must Be Layered

```

Business Identity
-> user, group, role, department
Device Identity
-> hostname, certificate, MAC, serial, posture
Session Identity
-> switchport, WLAN/VPN session, IP, SGT
Process Identity
-> executable, hash, signer, parent, command line
Agent Identity
-> AI agent name, framework, capabilities, tools
Flow Identity
-> source process, destination, protocol, DNS, TLS/SNI
Request Identity
-> API token, mTLS cert, model call, URL, resource action
Risk Identity
-> behavior, anomaly, privilege, data sensitivity

```

Design rule: SGT belongs at the device/session layer. Process identity, agent identity, and request identity require host-native and application-layer controls.

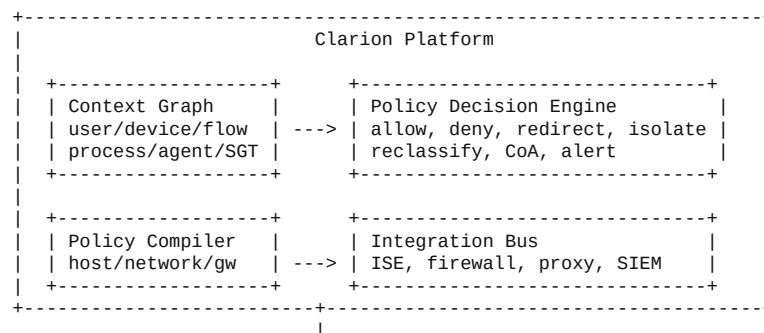
4.2 Trust Decisions Should Be Evidence-Based

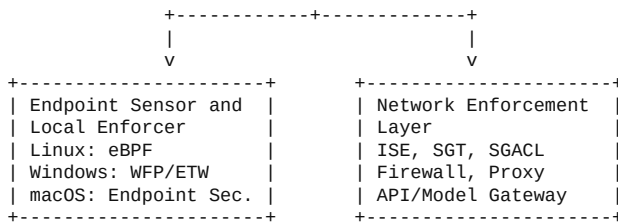
The endpoint sensor provides evidence, not final authority. Clarion validates the evidence, combines it with network and identity context, and then uses ISE, endpoint controls, gateway policies, or firewall integrations to enforce trusted decisions.

Bad model:
Endpoint says: I am SGT 10. Network trusts it.

Preferred model:
Endpoint sends signed evidence.
Clarion validates evidence.
ISE/network assigns trusted policy.
Switch propagates SGT northbound.

5. High-Level Architecture





6. Component Overview

6.1 Clarion Endpoint Sensor

The endpoint sensor collects local context and sends signed telemetry to Clarion. It discovers processes, tracks parent-child lineage, maps processes to flows, detects automation or agent-like behavior, collects DNS and destination context, and receives policy updates.

Platform	Visibility Mechanisms	Enforcement Mechanisms
Linux	eBPF, auditd, procfs, cgroups, systemd, DNS logs	eBPF, cgroup hooks, tc, nftables, local proxy
Windows	ETW, Sysmon, MDE, WMI, process APIs, DNS logs	WFP, Windows Firewall, local proxy, EDR actions
macOS	Endpoint Security Framework, Network Extension, unified logs	Network Extension, PF, local proxy, MDM controls

6.2 Clarion Local Enforcer

The local enforcer applies policy closest to the source process. It is the correct enforcement point when Clarion needs to block a single process, redirect a specific agent to a gateway, or prevent a developer tool from reaching a sensitive destination without disrupting the rest of the endpoint.

Example controls:

- Allow or deny a process-to-destination connection.
- Redirect agent traffic to an approved proxy or model gateway.
- Block direct database access from unauthorized tools.
- Block suspicious process chains such as encoded PowerShell with external beaconing.
- Cache policy for offline or temporarily disconnected enforcement.

6.3 Clarion Context Graph

The context graph correlates endpoint, identity, network, destination, and risk data. It should represent objects such as users, devices, network sessions, processes, applications, agents, flows, destinations, policy decisions, risk findings, and enforcement actions.

```

User Steve
-> Device dev-laptop-22
-> Network session Cat9300 Gi1/0/20
-> SGT Developer_Device
-> Process python.exe
-> Parent cursor.exe
-> Agent RepoResearchAgent
-> Destination github.com
  
```

6.4 Policy Decision Engine and Compiler

The policy decision engine determines what should happen. The policy compiler converts high-level intent into endpoint rules, network changes, gateway controls, firewall objects, or alerts.

High-Level Intent	Compiled Windows Example	Compiled Linux Example	Network/Gateway Example
AI agents may not directly access production databases	WFP deny for process/destination	eBPF/cgroup deny for agent process group	SGACL deny from AI_Active_Device to Production_DB
Browsers may not upload to unknown file-sharing sites	Firewall/proxy redirect by browser process	nftables/proxy redirect by process group	SWG/DLP policy block
Privileged tools require approved device posture	Local allow only when posture is healthy	Local allow only when attested	ISE SGT or dACL based on posture

7. Identity and Policy Model

7.1 Device-Level Identity

```
{
  "device_id": "dev-laptop-22",
  "hostname": "WIN-DEV-22",
  "mac": "AA:BB:CC:DD:EE:FF",
  "os": "Windows 11",
  "owner": "steve@example.com",
  "posture": "healthy",
  "switch": "Cat9300-Access-1",
  "interface": "Gi1/0/20",
  "current_sgt": "Developer_Device"
}
```

7.2 Process and Agent Identity

```
{
  "process": {
    "name": "python.exe",
    "path": "C:\\Users\\Steve\\agent_runner.py",
    "sha256": "abc123...",
    "publisher": "Python Software Foundation",
    "parent_process": "cursor.exe",
    "command_line": "python agent_runner.py --tools web,repo,llm"
  },
  "agent": {
    "name": "RepoResearchAgent",
    "framework": "autogen",
    "capabilities": ["repo_read", "web_access", "llm_call"],
    "confidence": 0.91,
    "risk_level": "medium"
  }
}
```

7.3 Flow Identity

```
{
  "flow_id": "flow-7821",
  "source_device": "dev-laptop-22",
  "source_process": "python.exe",
  "source_agent": "RepoResearchAgent",
  "destination": "github.com",
  "destination_category": "source_code_repository",
  "protocol": "tcp",
  "port": 443,
  "action": "allow"
}
```

8. SGT and TrustSec Strategy

8.1 SGT Represents Endpoint or Session State

SGTs should represent broad trusted network classes, not every process or agent. This is because multiple applications and agents can run simultaneously behind one access session and one access port.

SGT Name	Meaning	Typical Enforcement Use
Standard_User_Device	Normal managed user endpoint	Baseline user access
Developer_Device	Developer workstation	Access to development systems and approved repositories
Privileged_Admin_Device	Admin or jump-host tooling device	Restricted administrative paths
AI_Capable_Device	Device has approved AI tooling installed	Monitor and restrict sensitive direct access
AI_Active_Device	AI agent/tool activity detected	Permit only approved gateways and services
AI_Restricted_Device	AI behavior has elevated risk	Limit sensitive access and alert
Automation_Host	RPA, scripts, or automation host	Constrain automation paths
Quarantined_Device	High-risk endpoint	Isolation/remediation network

8.2 CMD Inline Tagging

CMD inline tagging should be used between TrustSec-capable network devices. The endpoint should send normal Ethernet frames. The access switch should assign the source SGT after authentication or policy classification and then propagate the SGT northbound using inline tagging when the fabric supports it.

```

Endpoint sends normal Ethernet frame
-> Access switch assigns source SGT
-> Access switch inserts CMD/SGT on uplink
-> Distribution/core/firewall consumes SGT
-> SGACL/firewall policy enforces

```

Clarion design position: Do not use endpoint-generated Cisco CMD headers as the primary design. Use endpoint evidence to influence trusted policy assignment through Clarion, ISE, switches, and enforcement systems.

9. Enforcement Architecture

Need	Best Enforcement Location
Per-process allow/deny	Endpoint
Per-agent allow/deny	Endpoint or proxy/model gateway
Per-device segmentation	ISE, access switch, SGT
East-west data center segmentation	Firewall, SGACL, workload policy
Internet/SaaS access control	SWG, SASE, proxy
LLM/API usage governance	Model gateway, API gateway
High-confidence quarantine	ISE CoA, SGT change, MDM/EDR
Suspicious behavior response	Endpoint plus SIEM/SOAR plus network

9.1 Enforcement Priority Model

Clarion should choose the least disruptive enforcement point that is precise enough for the risk. Blocking the entire endpoint is disruptive. If only one agent or process is risky, block that first.

1. Endpoint process-level block.
2. Endpoint process-level redirect.
3. Gateway/API/model-layer control.
4. Firewall dynamic object or app policy.
5. SGT reclassification.

6. VLAN or VRF quarantine.
7. Full endpoint isolation.

10. AI Agent and General Application Detection

This architecture supports AI agent detection, but it is not limited to AI agents. It should also classify and govern browsers, PowerShell, developer tools, database clients, RPA tools, local LLM runtimes, SSH clients, package managers, and suspicious automation.

10.1 AI Agent Signals

- Known AI agent frameworks and runtimes.
- LLM API destinations and model gateway usage.
- Repeated tool-call behavior and autonomous loops.
- Browser automation/headless browsing.
- Repository access followed by model calls.
- MCP server/client behavior.
- Vector database and local LLM access.
- Process lineage such as ``cursor.exe -> node.exe -> python.exe -> model API``.

10.2 General Application Examples

Category	Example Policy
Browser	Allow SaaS apps; block uploads to unknown cloud storage destinations.
PowerShell	Allow approved admin paths; block encoded external beaconing.
Developer tools	Allow approved repositories; block unknown external repo push.
Database clients	Allow dev DB; require privileged posture for production DB.
Automation/RPA	Allow approved SaaS workflows; block credential vault export endpoints.

11. Data Flows and Scenarios

11.1 Normal Flow

1. Endpoint connects to the network.
2. ISE authenticates user/device.
3. Switch assigns base SGT.
4. Clarion learns network session context.
5. Endpoint sensor reports process and flow context.
6. Clarion correlates process -> flow -> user -> device -> SGT.
7. Clarion evaluates policy.
8. Clarion pushes local, network, or gateway enforcement.

11.2 High-Risk Flow

1. Python process launches from Cursor.
2. Process reads local repository.
3. Process calls external model API.
4. Process attempts to connect to production database.
5. Endpoint sensor reports event to Clarion.
6. Clarion classifies process as AI agent with high confidence.
7. Clarion blocks locally through WFP/eBPF/Network Extension.
8. Clarion optionally changes endpoint SGT to AI_Restricted_Device.
9. Firewall/SGACL prevents further sensitive access.
10. SOC alert is created with full context.

11.3 Multiple Agents on One Endpoint

Device: Steve-Win11-Dev
User: Steve
Switchport: Cat9300 Gi1/0/20
Base SGT: Developer_Device

Running processes:
chrome.exe
cursor.exe
python.exe running RepoResearchAgent
node.exe running MCP filesystem server
docker container running LocalDataAgent
powershell.exe

Clarion action:
Block risky AI agent flow locally.
Keep normal browser traffic allowed.
Reclassify endpoint to AI_Active_Device only if risk or policy requires it.

12. Cisco ISE and TrustSec Integration

Clarion should integrate with ISE and TrustSec as a context and orchestration layer, not as a replacement for ISE. ISE remains the authoritative system for access authentication, authorization, SGT assignment, CoA, and session policy delivery to the switch.

Integration Area	Clarion Role	ISE/Switch Role
Endpoint context	Provide risk, posture, and behavior context	Consume context and authorize session
SGT recommendation	Recommend state such as AI_Active_Device	Assign SGT through authorization policy
CoA trigger	Request reauth or policy update after risk change	Send CoA and update access session
TrustSec propagation	Reason about policy and scope	Switch propagates SGT using inline tagging
Policy analytics	Recommend SGACLs and detect gaps	Enforce supported SGACL/dACL/firewall rules

13. Telemetry Sources

Source	Examples
Endpoint	Process start/stop, parent-child lineage, hashes, signatures, network connections, DNS, TLS/SNI, container identity, user session, enforcement events
Network	IPFIX/NetFlow, SGT source/destination, switchport/MAC/IP mapping, RADIUS accounting, ISE sessions, pxGrid, firewall logs, proxy logs, DNS, DHCP
Application/Gateway	API calls, model calls, prompt metadata, token usage, mTLS identity, OAuth client identity, URL category, DLP findings

14. Data Model and APIs

14.1 Core Collections/Tables

devices
users
network_sessions
processes
agents
flows
destinations
policies
policy_decisions
enforcement_actions

risk_findings
identity_mappings
sgt_mappings
integrations

14.2 Example Policy Decision Object

```
{  
  "decision_id": "dec-991",  
  "policy_id": "block_ai_agents_from_prod_db",  
  "subject": {  
    "device": "dev-22",  
    "process": "proc-9324",  
    "agent": "agent-001"  
  },  
  "destination": "prod-db.internal",  
  "decision": "deny",  
  "enforcement_points": ["endpoint_wfp", "network_sgt"],  
  "reason": "AI agent attempted direct production database access"  
}
```

14.3 API Surfaces

API Direction	Representative Endpoints
Endpoint sensor to Clarion	`POST /telemetry/process`, `/telemetry/flow`, `/telemetry/dns`, `/telemetry/agent-detection`, `/enforcement/status`
Clarion to endpoint sensor	`GET /policy/endpoint/{device_id}`, `POST /enforcement/apply`, `/enforcement/remove`, `/sensor/command`
Clarion to ISE	`POST /integrations/ise/endpoint-context`, `/change-sgt`, `/coa`; `GET /integrations/ise/sessions`
Clarion to gateway/firewall	`POST /integrations/firewall/dynamic-object`, `/integrations/proxy/policy`, `/integrations/model-gateway/policy`

15. Security and Trust Model

- Endpoint telemetry must be signed and protected with mutual TLS.
- Policy bundles must be signed and verified by the endpoint.
- Clarion must defend against replay with nonces, timestamps, and sequence IDs.
- The endpoint sensor must report tamper events such as service stop, driver unload, policy disablement, binary modification, or certificate failure.
- Severe tamper events may trigger restricted SGT assignment, SOC alert, and MDM/EDR response.

Critical security principle: Endpoint evidence can influence trusted network policy, but it should not directly assert trusted network tags. Clarion validates the evidence and then orchestrates policy through trusted enforcement systems.

16. Deployment Model

Layer	Components
Endpoint	Clarion Sensor Service, Clarion Enforcement Service, platform-specific hooks/drivers, local policy cache, secure telemetry channel
Backend	Ingestion API, policy engine, context graph, risk engine, policy compiler, integration services, event bus, telemetry store, admin UI
Network	Cisco ISE, access switches, TrustSec fabric, firewalls, proxy/SWG, DNS, DHCP, SIEM/SOAR
Enterprise management	MDM/Intune/Jamf/GPO, EDR, certificate authority, secrets management

17. MVP Phases

Phase	Goal	Build Items	Output
1. Visibility	Identify process/application/agent context	Windows and Linux sensors, process inventory, flow mapping, DNS correlation, AI-agent detection, dashboard	Device -> process -> flow -> destination map
2. Local Enforcement	Block or redirect specific processes	Windows WFP/Firewall control, Linux eBPF/nftables, local policy cache, audit logs	Per-process and per-agent enforcement
3. Network Integration	Reflect endpoint intelligence into SGT/ISE	ISE connector, SGT recommendation, CoA trigger, session correlation	Dynamic endpoint posture reflected in network policy
4. Gateway Integration	Enforce request-level policy	Model gateway, proxy/SWG, API gateway, token/mTLS identity, DLP	Per-agent and per-request governance
5. Optimization	Recommend and refine policy	Baselining, simulation, SGACL suggestions, natural language policy assistant	Clarion as policy intent and optimization system

18. Engineering Backlog

18.1 Endpoint Sensor Backlog

- Implement signed device enrollment and certificate rotation.
- Collect process start/stop, parent PID, command line, hash, signer, and user SID/UID.
- Map flows to process IDs and destination DNS categories.
- Detect AI-agent patterns using configurable signatures and behavior rules.
- Implement local policy cache and offline behavior.
- Report tamper and health state.

18.2 Policy Engine Backlog

- Build context graph entities and relationships.
- Implement policy evaluation pipeline with deterministic explanations.
- Create enforcement priority model.
- Add policy simulation before enforcement.
- Add exception handling and approval workflows.
- Expose policy authoring API and UI.

18.3 Integration Backlog

- ISE session correlation by MAC, IP, username, and switchport.
- SGT recommendation and endpoint group update workflow.
- CoA trigger with rollback and safety controls.
- Firewall dynamic object integration.
- Proxy/model gateway policy integration.
- SIEM/SOAR event export.

19. Final Design Summary

Clarion is a context-aware policy intelligence platform that combines endpoint process identity, network session identity, SGT-based segmentation, and application-layer enforcement to determine who or what should communicate with whom.

- SGTs provide coarse trusted network segmentation.
- Endpoint sensors provide fine-grained process and agent attribution.
- Local enforcement handles process-specific control.
- Gateways handle request-specific governance.
- Clarion correlates all context and orchestrates policy across the full path.

Final recommendation: Do not force SGT to solve per-process identity. Use SGT for network trust zones. Use host-native controls for process and agent identity. Use Clarion as the policy brain that ties them together.